Logging in Go

The standard library's three loggers (`log` , `log/slog`), the popular third-party choices (`zap` , `zerolog` , `logrus`), and how to do logging well in production Go.

The landscape

Package	What it is	When to use
`log`	Stdlib, unstructured, single global logger	Tiny tools, throwaway scripts
`log/slog`	Stdlib, structured (Go 1.21+)	Default for new code
`zap`	Uber, structured, fastest	Hot paths, high-volume logging
`zerolog`	Structured, zero-allocation JSON	Similar use case to zap, simpler API
`logrus`	Structured, mature, slower	Legacy projects (now in maintenance mode)

The 2024+ recommendation: use `slog` unless benchmarks show you need `zap` or `zerolog` . slog has multiple backends, including a zap adapter.

The stdlib `log` package

```go
import "log"

log.Println("hello")
log.Printf("user=%d action=%s", id, action)
log.Fatal("boom")     // logs then calls os.Exit(1)
log.Panic("ahh")      // logs then panics
```

Defaults: - Writes to `stderr` . - Prefix: date and time. - Format flags: `log.LstdFlags` (date + time).

Customize:

```
log.SetOutput(os.Stdout)
log.SetPrefix("[myapp] ")
log.SetFlags(log.LstdFlags | log.Lshortfile)  // also include "file.go:42"

logger := log.New(os.Stderr, "[req] ", log.LstdFlags|log.Lmicroseconds)
logger.Println("incoming")
```

What it doesn't do: - No log levels (info, warn, error). - No structured fields. - Concurrency: each `Print*` call is one atomic write, but no batching.

For new code, skip straight to slog.

---

log/slog

`log/slog` landed in Go 1.21. It's structured by default, integrated with `context.Context`, and supports multiple output handlers (text, JSON, custom).

Basic use

```
import "log/slog"

slog.Info("user logged in", "user_id", 42, "ip", "1.2.3.4")
slog.Warn("disk low", "free_mb", 100)
slog.Error("query failed", "err", err, "query", q)
slog.Debug("cache hit", "key", k)
```

Output (default text handler):

```
2026/05/24 12:00:00 INFO user logged in user_id=42 ip=1.2.3.4
```

The key-value pairs after the message are structured attributes.

Switching to JSON

```
logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))
slog.SetDefault(logger)
```

Now everything outputs JSON:

```
{"time":"2026-05-24T12:00:00Z","level":"INFO","msg":"user logged
        in","user_id":42,"ip":"1.2.3.4"}
```

JSON output is the production default. It's machine-parseable, plays well with log aggregators (Datadog, Loki, ELK), and preserves types.

Levels

`slog.Level` is an int8. Predefined: `LevelDebug` (-4), `LevelInfo` (0), `LevelWarn` (4), `LevelError` (8). Custom levels in between are supported.

Set the minimum level on the handler:

```
opts := &slog.HandlerOptions{Level: slog.LevelWarn}
logger := slog.New(slog.NewJSONHandler(os.Stdout, opts))
```

Below the minimum, log calls are skipped cheaply (no allocations).

Dynamic level

For flipping levels at runtime (admin endpoint, signal handler):

```
var level = new(slog.LevelVar)        // defaults to Info
level.Set(slog.LevelDebug)

logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{Level:
        level}))
```

Change `level.Set(...)` from anywhere; new log calls respect it immediately.

Attributes

The key-value pairs after the message are “attributes.” There are two ways to pass them:

```
// shorter form: alternating key, value
slog.Info("x", "key", value, "key2", value2)

// typed form: avoids reflection cost
slog.Info("x", slog.Int("user_id", 42), slog.String("action", "login"))
```

The typed form is faster and catches typos at compile time. Slog auto-uses the right type when you pass an `slog.Attr`.

Available constructors: `slog.String`, `slog.Int`, `slog.Int64`, `slog.Uint64`, `slog.Float64`, `slog.Bool`, `slog.Duration`, `slog.Time`, `slog.Any`, `slog.Group`.

Groups (nested attributes)

```
slog.Info("http request",
    slog.Group("req",
        "method", "GET",
        "path", "/api/users",
        "status", 200,
    ),
    "duration_ms", 42,
)
```

JSON:

```
{"msg":"http request","req":
        {"method":"GET","path":"/api/users","status":200},"duration_ms":42}
```

Logger with attached attributes

```
reqLogger := slog.Default().With("request_id", "abc123", "user_id", 42)
reqLogger.Info("processing")
reqLogger.Error("failed", "err", err)
```

`With` returns a new logger that includes those attributes on every call. Useful for request-scoped logging.

Context integration

```
slog.InfoContext(ctx, "x", "key", v)
```

The handler can extract attributes from the context. The default handler doesn't do this; custom handlers (and OTel integration) often do.

Custom handler

```
type myHandler struct{}

func (h *myHandler) Enabled(ctx context.Context, level slog.Level) bool { ... }
func (h *myHandler) Handle(ctx context.Context, r slog.Record) error    { ... }
func (h *myHandler) WithAttrs(attrs []slog.Attr) slog.Handler           { ... }
func (h *myHandler) WithGroup(name string) slog.Handler                 { ... }
```

Write a handler when you need to: - Send logs over the wire (syslog, gRPC log shipper). - Add per-request context (trace ID, user ID) automatically. - Sample logs at high volume.

---

zap (third-party, ultra-fast)

```
import "go.uber.org/zap"

logger, _ := zap.NewProduction()    // JSON, info level, with sampling
defer logger.Sync()

logger.Info("user logged in",
    zap.Int("user_id", 42),
    zap.String("ip", "1.2.3.4"),
)

sugar := logger.Sugar()
sugar.Infow("user logged in", "user_id", 42, "ip", "1.2.3.4")
sugar.Infof("user %d logged in", 42)
```

Two APIs: - **Logger:** strict, typed, fastest. Use for hot paths. - **SugaredLogger:** loose, key-value pairs and printf-style. Slower but ergonomic.

Why zap is fast

- Pre-allocated buffers.
- No reflection for typed field constructors (`zap.String` , `zap.Int` , ...).
- Encoder writes directly into buffer; no intermediate map.

Benchmark: zap is roughly 5-10x faster than logrus, 2-3x faster than slog with the JSON handler (varies by workload and Go version).

Sampling

`zap.NewProduction()` enables sampling: when many identical log lines fire in a short window, most are dropped. Prevents log floods from crashing your aggregator.

```
cfg := zap.NewProductionConfig()
cfg.Sampling = &zap.SamplingConfig{Initial: 100, Thereafter: 100}
logger, _ := cfg.Build()
```

After the first 100 in a second, drop all but every 100th.

---

zerolog (third-party, zero-allocation)

```
import "github.com/rs/zerolog/log"

log.Info().
    Int("user_id", 42).
    Str("ip", "1.2.3.4").
    Msg("user logged in")

log.Error().Err(err).Str("query", q).Msg("query failed")
```

Chain-call API. Each `.Type(key, val)` adds a field; `.Msg(...)` finishes and writes the line.

Similar performance to zap. Some find the chain API more readable; others prefer slog's variadic form.

Comparison: writing the same line

```
// log (stdlib, unstructured)
log.Printf("user logged in user_id=%d ip=%s", id, ip)

// slog
slog.Info("user logged in", "user_id", id, "ip", ip)

// slog typed
slog.Info("user logged in", slog.Int("user_id", id), slog.String("ip", ip))

// zap
logger.Info("user logged in", zap.Int("user_id", id), zap.String("ip", ip))

// zap sugared
sugar.Infow("user logged in", "user_id", id, "ip", ip)

// zerolog
log.Info().Int("user_id", id).Str("ip", ip).Msg("user logged in")

// logrus
logrus.WithFields(logrus.Fields{"user_id": id, "ip": ip}).Info("user logged in")
```

Structured logging principles

Pick consistent keys

Decide once whether you use `user_id` or `userId` or `user-id`. Stick with it. Aggregator queries depend on it.

Common conventions: - snake_case keys. - Lowercase. - Singular nouns. - Suffix units: `_ms`, `_bytes`, `_count`.

Don’t log secrets

```
// BAD
slog.Info("login", "user", username, "password", pw)

// BAD (might log the whole user)
slog.Info("login", "user", userStruct)   // userStruct.PasswordHash leaks
```

Use a `Redact` helper or implement `LogValue` on the struct:

```
type User struct {
    ID    string
    Email string
    PW    string
}

func (u User) LogValue() slog.Value {
    return slog.GroupValue(
        slog.String("id", u.ID),
        slog.String("email", redact(u.Email)),
    )
}
```

slog calls `LogValue()` when serializing.

Log at the right level

Level	Use for
Debug	Detail useful when investigating; off in prod
Info	Normal events worth recording (request handled, job complete)
Warn	Something looks off but the system kept going (retry, fallback)
Error	An operation failed; needs attention or automated alert
(Fatal/Panic)	Process can’t continue; let it die

Resist “info everywhere.” High-volume info logs cost money and obscure real events.

Errors as values, not strings

```
// BAD
slog.Error("failed: " + err.Error())

// GOOD
slog.Error("query failed", "err", err)
```

Structured `err` lets aggregators index it, group similar errors, and link to traces. The string concat form throws that away.

Include enough to debug, no more

The 5 W's of a log line: who (user, request), what (action, error), when (timestamp, automatic), where (function, automatic), why (context). If you couldn't reproduce a bug from the log alone, add what's missing.

Per-request logger

Attach the request ID once, log everywhere within the handler:

```
func middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        reqID := r.Header.Get("X-Request-ID")
        if reqID == "" { reqID = newID() }
        logger := slog.Default().With("req_id", reqID)
        ctx := context.WithValue(r.Context(), loggerKey{}, logger)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

func loggerFromCtx(ctx context.Context) *slog.Logger {
    if l, ok := ctx.Value(loggerKey{}).(*slog.Logger); ok { return l }
    return slog.Default()
}
```

Output destinations

stdout / stderr

The 12-factor default. Let the process write to stdout; let the platform (systemd, Docker, k8s) collect and route.

```
logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))
```

File

Don't write directly. Use a rotating logger:

```
import "gopkg.in/natefinch/lumberjack.v2"

w := &lumberjack.Logger{
    Filename:   "/var/log/app.log",
    MaxSize:    100,  // MB
    MaxBackups: 3,
    MaxAge:     28,   // days
    Compress:   true,
}

logger := slog.New(slog.NewJSONHandler(w, nil))
```

Lumberjack rotates the file by size and keeps a bounded archive.

Network (syslog, GELF, Loki, etc.)

Use a handler that ships logs over the wire. Most agents (Fluent Bit, Vector, Datadog Agent) prefer reading stdout and forwarding, which is simpler than in-process shipping.

Performance

Approximate cost per log call (Go 1.21, 64-bit, simple message + 3 fields):

Logger	Per-call cost	Allocations
`log.Printf` (unstructured)	~700 ns	1-2
`slog` JSON, typed attrs	~300 ns	0-1
`slog` JSON, untyped (any)	~600 ns	2-4
`zap` (production, typed)	~120 ns	0-1
`zerolog` JSON	~100 ns	0
`logrus` JSON	~3000 ns	20+

Numbers vary. Run `go test -bench .` against your workload. For most apps, slog is enough. For services logging 100K+ lines/sec, switch to zap or zerolog.

Costs to watch for

- **Reflection.** Untyped attribute keys (any value) cost reflection per call.

- **Pre-formatting messages that get dropped.** `slog.Debug(fmt.Sprintf(...))` formats even if debug is off. Use `slog.Debug("x", "key", value)` so the value is captured lazily.
- **String concatenation in messages.** Build the structure, let the encoder do the work.

Log shipping and aggregation

Once logs are on stdout as JSON, ship them to a backend:

Backend	Notes
Datadog	Agent reads stdout, parses JSON
Loki + Grafana	Promtail/Vector tailers; cheap storage
Elasticsearch	Filebeat/Fluentd ingest
CloudWatch	AWS Lambda / ECS automatically forwards
GCP Cloud Logging	Same on GCP
Splunk	Universal Forwarder

Pattern in k8s: app writes JSON to stdout, a sidecar (Fluent Bit) tails the container log file and ships to the backend. The app doesn't know or care about the destination.

Correlating with traces

If you're running OpenTelemetry (see tracing sheet), pull the trace ID into every log line:

```
import "go.opentelemetry.io/otel/trace"

span := trace.SpanFromContext(ctx)
logger := slog.Default().With(
    "trace_id", span.SpanContext().TraceID().String(),
    "span_id",  span.SpanContext().SpanID().String(),
)
```

Now any log line tied to that request can be looked up by trace ID in the aggregator. Click from a slow trace to its log lines and back.

Best practices

1. **Use slog for new code.** Stdlib, structured, multi-backend.
2. **Always JSON in production.** Text logs are fine for local dev.
3. **Log errors with the error as a structured value.** `"err", err`.
4. **Set a sane minimum level.** Info in prod, debug in staging.
5. **Attach a request ID to every request and propagate via context.**
6. **Never log secrets.** Implement `LogValue` for types that might be tempted.
7. **Don’t log in tight loops.** Aggregate first, log once.
8. **Sample if you can’t slow down.** Zap’s sampling is good prior art.
9. **Use lumberjack for files, stdout otherwise.** Let the platform do the rest.
10. **Add trace IDs to logs.** Correlation is what makes logs useful.

Common gotchas

Gotcha	Symptom	Fix
`log.Fatal` skips defers	Cleanup doesn't run	Bubble error up, exit at main
Logging in handler doesn't include request ID	Useless in production	Use context-scoped logger
Debug messages built with fmt.Sprintf	Allocates even when dropped	Pass key/value pairs; defer formatting
Default text format in prod	Aggregator can't parse	Switch to JSON handler
Logging a struct with sensitive fields	Secrets leak	Implement `LogValue`; redact explicitly
Multiple goroutines write the same file	Interleaved output	Use lumberjack or a serializing wrapper
Logging in hot loop kills perf	High CPU, log floods	Move log out of loop or sample
Untyped attrs in slog	Reflection cost	Use typed: `slog.Int`, `slog.String`
Log level set at startup, can't change	Stuck at info when debugging	Use `slog.LevelVar` with admin endpoint
Trace ID missing from log lines	Can't correlate	Add via middleware before handler runs
Logrus on a hot path	Slow, GC pressure	Migrate to slog/zap
Pretty-printing JSON	Larger size, slower parse	Single-line compact JSON in prod

Quick reference

What	Code
Stdlib unstructured	`log.Println("...")`
Slog JSON to stdout	`slog.New(slog.NewJSONHandler(os.Stdout, nil))`
Slog text to stderr	`slog.New(slog.NewTextHandler(os.Stderr, nil))`
Set default logger	`slog.SetDefault(logger)`
Set min level	`&slog.HandlerOptions{Level: slog.LevelWarn}`
Dynamic level	`slog.LevelVar`
Typed attribute	`slog.Int("k", v)`, `slog.String(...)`, …
Logger with attrs	`slog.Default().With("req_id", id)`
Group	`slog.Group("req", "method", "GET", ...)`
Context-aware log	`slog.InfoContext(ctx, "...", ...)`
Custom redaction	`LogValue()` `slog.Value` on type
File rotation	`lumberjack.Logger`
Zap production logger	`zap.NewProduction()`
Zap typed field	`zap.Int("k", v)`
Zap sugared	`sugar.Infow("msg", "k", v)`
Zerolog	`log.Info().Str(...).Msg("...")`
Trace correlation	Add `trace_id` and `span_id` via context